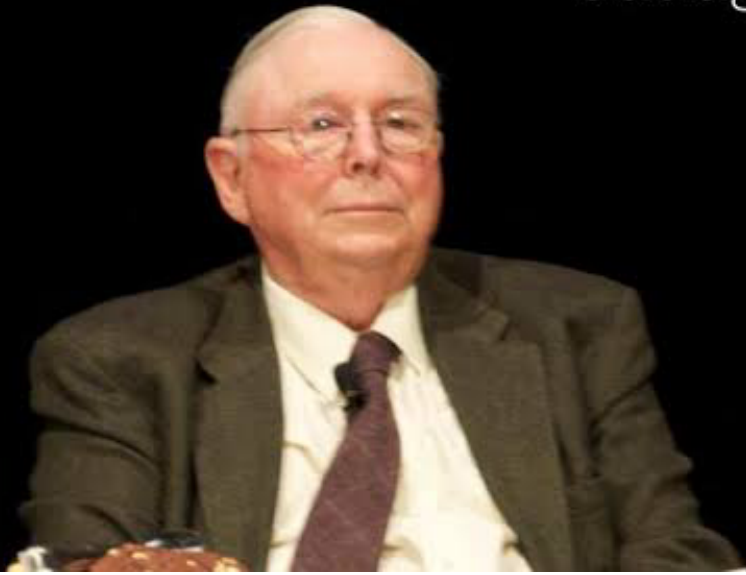


“Success means being very patient,
but aggressive when it's time.”



— Charlie Munger

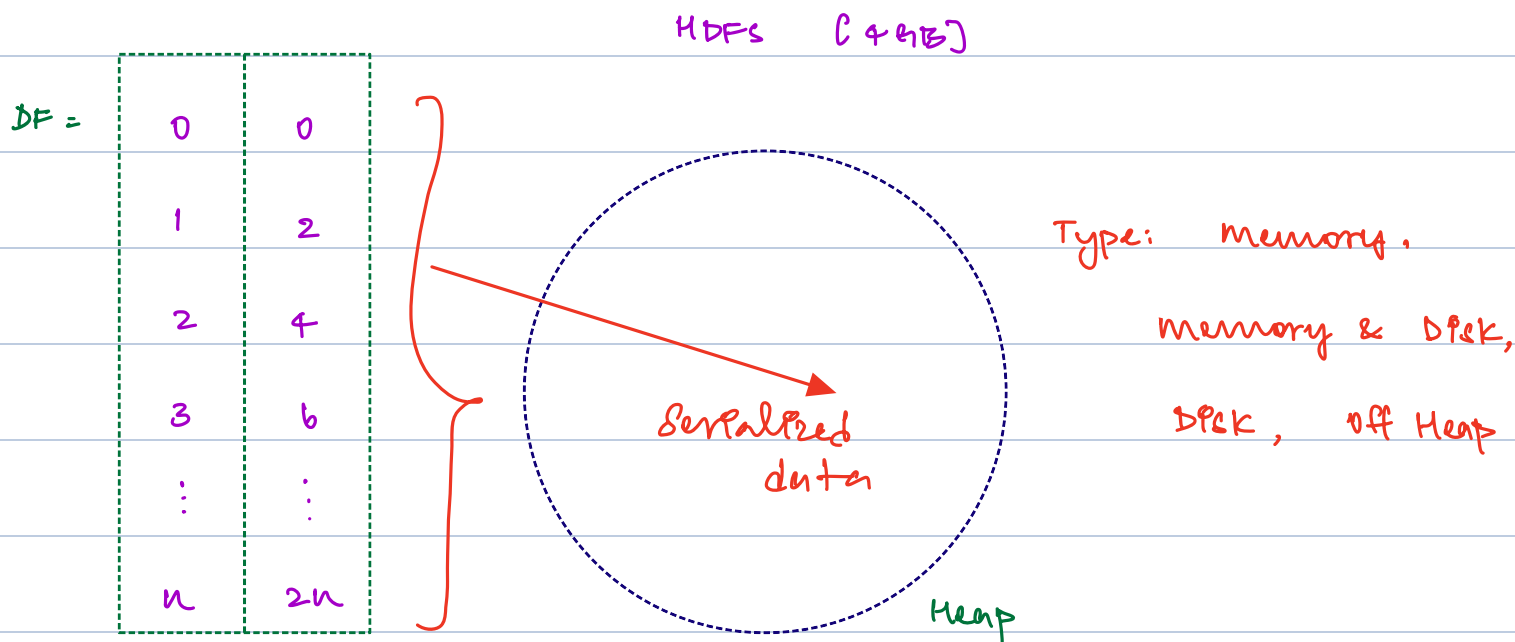
Agenda

- ① Revision
- ② Data Frame caching & Persist [UDF]
- ③ Broadcasting in Join
- ④ Salting, Repartition and coalesce
- ⑤ Adaptive Query Execution [AQE]
- ⑥ Advanced configs

If time permits

- ① Interview question

Dataframes Caching & Persist → avoids recomputation



Fraction cached

Partitioned cached

Memory used

unpersist ()

create DF → Filter → Join → parquet

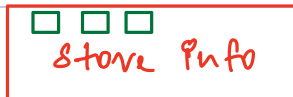
DF → Filtered → Aggregation → write.

DF → Filter (2) → Join → Aggregate → write

cache

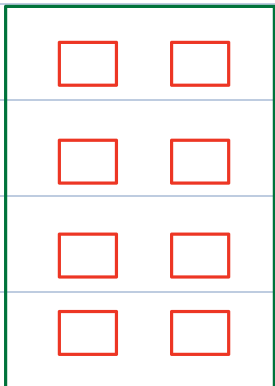
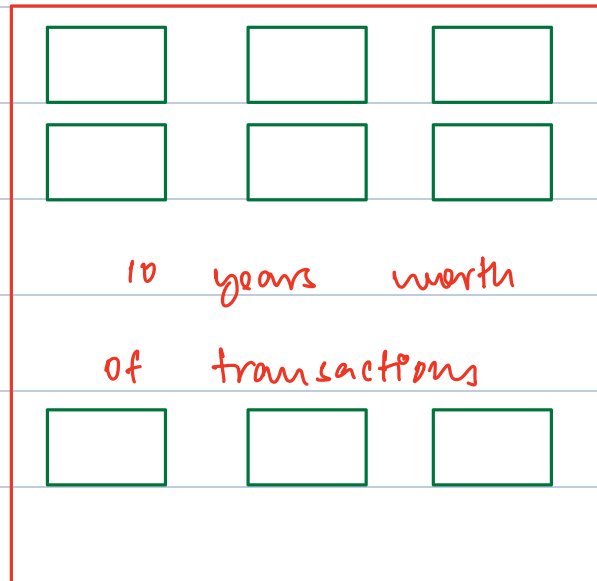
Broadcasting

Sharing a piece of data to all participants
in Full Data Frame partitions



Base join

partition of transaction
partition of store info



small DF

WIDE
TRANSFORM

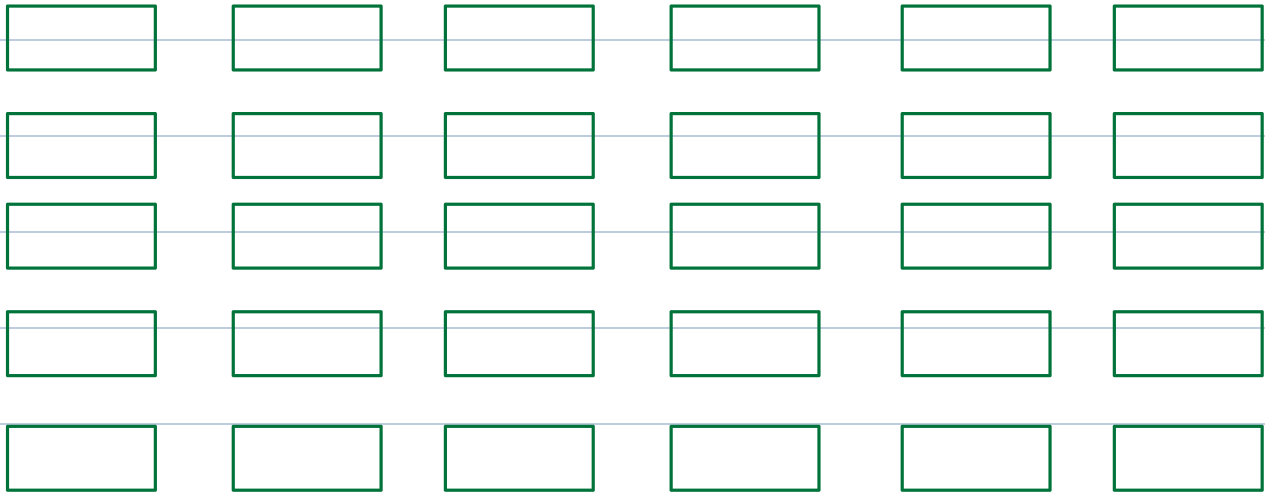
ID



larger DF

spark creates 200 partition by default

spark.sql.shuffle.partition



Data with similar ID should come to a single partition for me to perform Join

190 empty partition

10 non-empty partition

data skew

} add a column

How to fix

Broadcast smaller DF to all partitions

small DF
(partition)

8

driver → all Data executors

Default value → 10 MB

spark.sql.autoBroadcastThreshold

The actual limitation of Broadcasting depends on your executor size

Coalesce

200 partitions after shuffle

10 partitions which had data

190 partitions did not have any data

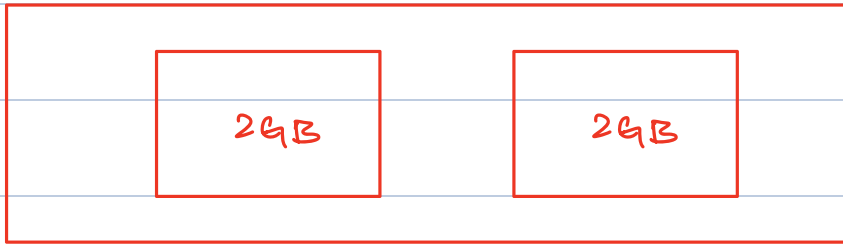
To reduce number of partition coalesce
The major benefit is, it reduces the
unnecessary cost of underused cores

Future : use ABE

adaptive coalesce partitions

Repartition

Increase number of partitions

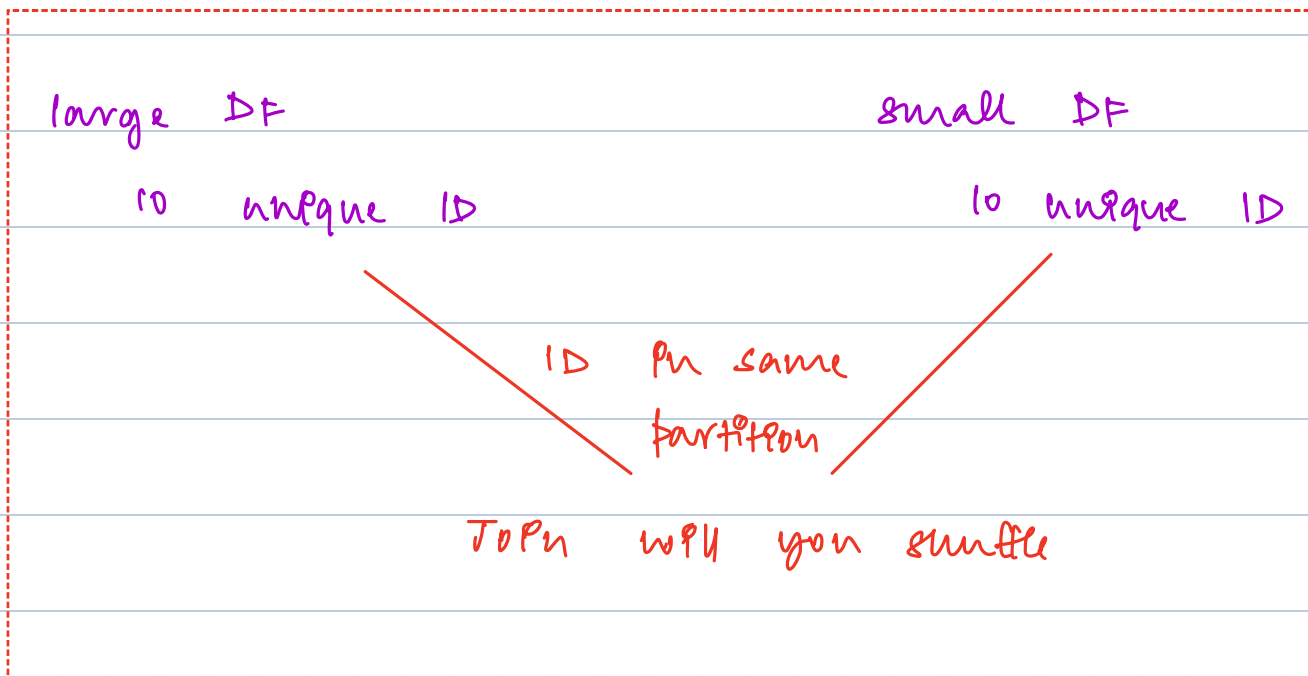


Increase parallelism

Improve data distribution

repartition (no. of partition)

repartition (column)



Home
work